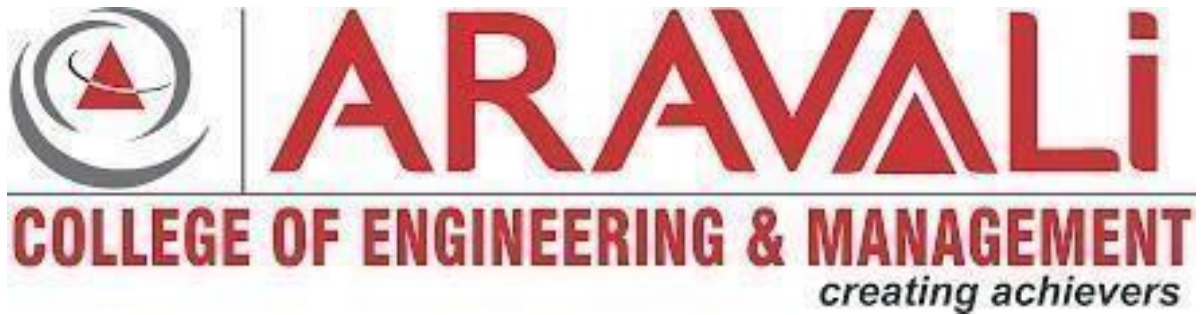




PROJECT REPORT

ON

EDU CONTROL ERP

DEPARTMENT OF CSE

ARAVALI COLLEGE OF ENGINEERING AND MANAGEMENT
FARIDABAD-121002

Submitted To:

Mrs. EKTA GARG

(Assistant Professor)

Submitted By:

Ali Mohammad

BCA DS 4th A

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the guidance, support and encouragement of several individuals who contributed their valuable time and expertise throughout every phase of development. We take this opportunity to express our heartfelt gratitude to all of them.

First and foremost, we would like to express our sincere thanks to our project guide, whose constant supervision, patient guidance and constructive feedback shaped the direction of this work. The technical insights offered during weekly reviews helped us overcome many implementation bottlenecks, especially while designing the file-upload pipeline that handles streaming uploads to Cloudinary and the JWT authentication flow that secures every protected route.

We extend our gratitude to the Head of the Department of BCA (Data Science) at Aravali College of Engineering and Management, Faridabad, for providing the academic environment, laboratory resources and departmental support needed to carry out a project of this scale. We also thank the entire faculty of the BCA (DS) department, whose coursework on web technologies, database systems, software engineering, JavaScript programming and data structures formed the conceptual foundation for this Smart Cloud Storage System.

We gratefully acknowledge the developers and maintainers of the open-source tools and services that made this project possible: the React and Vite teams for the lightning-fast frontend tooling, Tailwind CSS for the utility-first styling system, Node.js and Express for the backend runtime, MongoDB and Mongoose for the document database and ODM. Without these tools, building a production-grade cloud storage application within a single semester would have been impossible.

We are deeply thankful to our classmates and friends, who acted as early beta testers, uploaded their actual study material to stress-test the system, and reported bugs ranging from edge cases in the trash-restore flow to subtle inconsistencies in the dark-mode colour palette. Their honest criticism on everything from the file-preview modal to the bulk-selection UX directly improved the final product.

Finally, we would like to thank our families for their unwavering support, patience and encouragement during long coding sessions and late-night debugging marathons. Their belief in us kept the momentum going through every phase of this project.

Ali Mohammad

Roll No: 24011312005

BCA (Data Science) -4 Sem

ABSTRACT

EduControl is a comprehensive, React-based school management portal designed as a multi-role academic management system. The primary objective of the project is to consolidate the administrative and academic activities of a school — student enrolment, faculty management, classroom communication and assignment delivery — into a single, unified, easy-to-use web application that works reliably across modern browsers.

Key features include a role-based authentication system supporting Student, Faculty and Admin logins; an Admin panel for managing students and faculty records; a real-time classroom feed where teachers can post notes and PDF assignments that students can download; a personal profile dashboard showing roll number, department, assigned teacher and join date; and persistent local storage ensuring data survives browser refresh. The methodology followed an iterative, component-driven development cycle — requirement analysis, prototyping, progressive enhancement and user testing. Results show a clean, responsive interface across desktop and mobile browsers, seamless PDF upload and download functionality, and stable state management using React Context API.

Keywords: School Management System, React.js, Role-Based Authentication, Classroom Management, Context API, Local Storage, Tailwind CSS, Single Page Application.

Table of Contents

CHAPTER 1 — INTRODUCTION	1
Major Problems Identified	12
Proposed Solution by EduControl	12
1.4.1 Technical Feasibility	15
1.4.2 Economic Feasibility	17
1.4.3 Operational Feasibility	17
CHAPTER 2 — LITERATURE REVIEW	18
2.1 Existing School Management Systems	18
2.3 Role-Based Access Control (RBAC) in Web Applications	19
2.4 Local Storage as a Lightweight Persistence Layer	20
CHAPTER 3 — SYSTEM ANALYSIS	21
3.4.1 Performance Requirements	24
3.4.2 Safety Requirements	26
3.4.3 Security Requirements	26
CHAPTER 4 — SYSTEM DESIGN	27
4.2 Component Structure	29
4.3 State Management	30
4.4 Data Flow	32
CHAPTER 5 — SYSTEM IMPLEMENTATION	33
5.1.1 Authentication Module (Login Component)	34
5.1.2 Server Bootstrap (server.js)	36
5.1.3 Admin Module	37
5.1.4 Classroom Module	38
5.5 Dashboard Module	40
5.4 Frontend Dependencies	41
5.4.1 Deployment	42
CHAPTER 6 — RESULTS AND DISCUSSION	43
6.1 Test Environment	43
6.2 Functional Test Results	43
6.3 Performance Analysis	
6.4 Discussion	
CHAPTER 7 — SUMMARY AND CONCLUSION	44
7.1 Limitations of the System Conclusion	44
7.1 Future Scope	44
REFERENCES	45

LIST OF FIGURES

S.no	Chapter no.	Image name
1	CHAPTER 4 SYSTEM DESIGN	<i>Figure 4.3 — Dashboard — File Grid Layout (live screenshot)</i>
2	CHAPTER 5 SYSTEM IMPLEMENTATION	<i>fig 5.1-Project Structure</i> <i>fig 5.1.3.1- Student code Initialization</i> <i>fig 5.1.3.2- Faculty code Initializaion</i> <i>Table 5.1.4- Class Module</i>
3	CHAPTER 6 RESULTS AND DISCUSSION	<i>Table 6.1 — Functional Test Results Summary</i>

CHAPTER 1

INTRODUCTION

The rapid growth of technology in the education sector has transformed the way schools and colleges manage academic and administrative activities. Educational institutions are increasingly adopting digital systems to improve communication, maintain records efficiently, and provide better access to information for students, teachers, and administrators. However, many institutions still rely on traditional methods of management that involve manual paperwork, disconnected communication channels, and time-consuming administrative processes. These outdated systems often result in inefficiency, data redundancy, and difficulty in accessing important information when needed.

To overcome these challenges, there is a growing need for a centralized and user-friendly digital platform that can simplify educational management activities. EduControl has been developed as a modern solution to address these issues by providing a unified browser-based management system for different users within an institution. The application serves as a single-page application (SPA) where Admin, Faculty, and Student users can securely access their respective functionalities through one login portal.

EduControl is developed entirely using React.js and styled with Tailwind CSS to ensure a responsive and attractive user interface. The application also uses the React Context API for efficient global state management across different components. One of the key advantages of the system is that it does not require a dedicated backend server. All application data is stored in the browser's local storage, making the system lightweight, cost-effective, and easy to deploy without additional infrastructure expenses.

The system provides separate modules for different stakeholders. The Admin module allows administrators to add, manage, and view details of students and faculty members along with timestamps for record tracking. The Faculty module enables teachers to upload classroom notes, share announcements, and attach PDF documents for academic purposes. Students can access these materials directly from their dashboard and download shared resources whenever required. This improves collaboration and ensures smooth communication between teachers and students.

The project aims to reduce paperwork, improve efficiency, and provide a convenient digital environment for educational management. By integrating multiple functionalities into a single platform, EduControl

enhances accessibility, transparency, and productivity within educational institutions. The system is designed to be simple, scalable, and easy to use for all users regardless of their technical expertise.

This chapter introduces the problem statement that motivated the development of EduControl and explains the objectives and scope of the project. It also discusses the feasibility study, including technical, operational, and economic feasibility aspects. The following chapters provide detailed information regarding literature survey, system analysis, system design, implementation, testing, results, conclusion, and future enhancements of the project.

1.1 Problem Statement

- Educational institutions today face many difficulties in managing academic and administrative activities efficiently. Most schools and colleges still use traditional methods such as manual registers, paperwork, and separate communication platforms for handling student records, assignments, and faculty management. These disconnected systems create confusion, inefficiency, and delays in day-to-day operations.
- There is no single platform that connects Admin, Faculty, and Students together with proper role-based access. Because of this, administrators struggle to maintain records properly, teachers face difficulty sharing study material, and students do not get a clear view of their academic information.
- Another major issue is communication. Teachers often depend on external applications such as email or messaging platforms to send assignments and notes. This process is not organized and can lead to students missing important classroom updates. In addition, students cannot easily view details such as assigned teachers, enrollment dates, or classroom notifications in one centralized system.
- Data management is also a challenge in modern web applications. Without proper storage mechanisms, data entered into a React application may disappear after refreshing the page. Security is another important concern because unauthorized users should not be able to access protected pages or sensitive information within the portal.
- EduControl has been developed to solve these problems by providing a single authenticated educational management system. It integrates all users into one platform and provides secure access according to their roles. The system allows teachers to upload notes and PDF assignments directly in the classroom feed, while students can access and download them

easily. Data persistence is maintained using browser local storage, ensuring that records remain safe even after page refreshes.

Major Problems Identified

- **Role Fragmentation:**

No unified system exists for Admin, Faculty, and Students with proper access control.

- **Communication Gap:**

Teachers cannot easily share assignments, notes, or PDFs directly with students inside one platform.

- **Manual Record Management:**

Administrators must maintain records manually, increasing workload and chances of errors.

- **Lack of Transparency:**

Students cannot clearly view assigned teachers, enrollment details, or academic updates.

- **Data Loss Risk:**

Without persistent storage, application data may disappear after page refresh.

- **Security Issues:**

Unauthenticated users may gain access to protected routes without proper authentication systems.

- **Absence of Audit Trail:**

There is no automatic timestamp or tracking system for records and activities.

Proposed Solution by EduControl

- A centralized educational management portal for all users.
- Secure login system with role-based authentication.
- Faculty classroom feed for notes and PDF sharing.
- Student dashboard with complete academic visibility.
- Local storage integration for persistent data management.

- Responsive and user-friendly interface using React.js and Tailwind CSS.
- Simplified administration with automated timestamps and organized records.

1.2 Objective of the Proposed System

The primary objectives of the EduControl System are:

- To design and develop a unified, browser-based educational management system that integrates Admin, Faculty and Student functionalities behind a single responsive portal.
- To provide a secure credential-based authentication mechanism that validates user identity before granting access to protected dashboards and routes.
- To build dedicated role-based dashboards where each user — Admin, Faculty and Student — can access only the information and features relevant to their responsibilities.
- To implement an Admin module that supports adding, viewing and managing student and faculty records with automatically generated date-and-time stamps for better record tracking.
- To develop a classroom feed system that enables faculty members to post text notes, announcements and PDF assignments directly for students within the same department.
- To provide students with a centralized academic interface where they can view classroom updates, download study material and access information about assigned teachers.
- To create a detailed personal profile dashboard displaying role-specific information such as roll number, employee ID, department, assigned faculty member and joining date.
- To ensure complete data persistence across browser sessions using local storage so that records, classroom posts and user information remain available after page refreshes.
- To implement department-based visibility control so that classroom content is accessible only to authorized students belonging to the same department.
- To improve communication and transparency between administrators, teachers and students through a single integrated platform instead of multiple disconnected systems.

- To provide a lightweight and cost-effective architecture that operates entirely on the client side without requiring a dedicated backend server or database infrastructure.
- To maintain a clean, modular and scalable code structure using React.js, Tailwind CSS and React Context API for easier maintenance and future feature expansion.
- To keep the entire system user-friendly, responsive and extensible with future enhancements such as cloud database integration, attendance management, online examinations and real-time notifications.

1.3 Scope of the Proposed System

The scope of the project covers the following functional and technical areas:

- Authentication and access control: role-based login for Students using name and roll number, Faculty using name and employee ID, and Admin using fixed credentials, with protected routes that prevent unauthorised access to the portal.
- Admin management: addition and management of student and faculty records including name, roll number or employee ID, department, subject and automatically generated date-and-time stamps for tracking purposes.
- Classroom communication: a classroom feed where faculty members can post text notes, announcements and PDF assignments, with department-based visibility so that students can access only the content relevant to their department while admins can monitor all classroom activity.
- Profile and dashboard system: personalised dashboards for every logged-in user displaying role-specific information such as student roll number, faculty employee ID, department, joining date and assigned faculty member details for students.
- Persistent data storage: storage and retrieval of all application data — including student records, faculty records, classroom posts and profile information — through browser local storage with automatic reloading after page refreshes.

- **User interface and experience:** a responsive single-page application with protected navigation, sidebar-based dashboard layouts, toast notifications, smooth transitions and user-friendly interaction built using React.js and Tailwind CSS.
- **State and data management:** centralized state handling using React Context API for maintaining authentication state, classroom updates and shared application data across components without requiring external state-management libraries.
- **System architecture:** a lightweight frontend-only educational management platform that operates completely inside the browser without dependence on a dedicated backend server, database infrastructure or cloud hosting services.
- **Extensibility and maintenance:** a modular and maintainable component-based structure designed to support future enhancements such as cloud database integration, attendance tracking, online examinations, real-time notifications and performance analytics.

The scope explicitly excludes: dedicated backend servers or database systems (the application is entirely frontend-based), real-time messaging and push notifications, live video conferencing or streaming features, and native Android or iOS mobile applications. These functionalities are identified as possible future enhancements in later phases of development.

1.4 Feasibility Study

A feasibility study was conducted before entering full-scale development to verify that the proposed system is achievable within available time, technology and cost constraints. Three dimensions were considered: technical, economic and operational feasibility.

1.4.1 Technical Feasibility

The project is technically feasible because every required component is either free, open-source software or natively supported by modern web browsers:

- **Programming language and framework:** JavaScript with React.js provides a component-based architecture for building a responsive and interactive single-page educational management application.

- Frontend tooling: Vite offers a fast development environment with hot-module replacement, quick build times and lightweight production optimisation suitable for modern React applications.
- Styling framework: Tailwind CSS enables rapid user-interface development through utility-first classes, responsive layouts and consistent design without requiring extensive custom CSS.
- Client-side routing: React Router v6 provides seamless navigation between Login, Dashboard, Classroom and Admin modules without requiring full page reloads or server round-trips.
- State management: React Context API together with hooks such as `useState` and `useEffect` supports centralized state handling, authentication flow and automatic UI updates without depending on external state-management libraries.
- Browser storage: the Local Storage API is natively available in all modern browsers and provides persistent client-side data storage for student records, faculty details and classroom posts without requiring a backend database.
- File handling support: the `FileReader` API allows PDF documents and classroom files to be converted into browser-readable formats for upload, preview and download functionality directly inside the application.
- Security and route protection: protected routing and credential validation mechanisms ensure that unauthorised users cannot access restricted dashboards or sensitive educational data.
- Deployment feasibility: because the entire system operates on the frontend without server-side infrastructure, the application can be deployed easily on free static hosting platforms with minimal configuration and zero database cost.

1.4.2 Economic Feasibility

The project is strongly economically feasible because the entire stack is built on free developer tiers and open-source software. Development costs are effectively zero beyond the students' own time, an existing personal computer and an internet connection.

- **Development hardware:** already owned (no incremental cost).
- **Software licences:** every framework and library is released under permissive open-source licences (MIT, Apache 2.0, ISC, BSD).
- **MongoDB Atlas:** free M0 cluster with 512 MB shared storage — more than sufficient for metadata of all uploaded files..
- **Hosting:** Vercel and Render free tiers cover frontend deployment with no monthly cost.
- **Maintenance:** primarily the developers' own time during academic semesters.

Even at moderate scale, projected operating costs remain dramatically lower than commercial alternatives such as Google Workspace (which charges per user per month), making the System a genuinely economical choice for individuals and small teams.

1.4.3 Operational Feasibility

Operational feasibility measures whether the proposed system will actually be adopted and effectively used by administrators, faculty members and students within an educational environment. Because EduControl is a browser-based application, users require nothing more than a modern web browser and a basic internet connection to access the system. The user interface follows conventions familiar from widely used educational and management platforms — a left-hand navigation sidebar for dashboard access, a central content area for classroom posts and records, and interactive forms for adding or managing information — ensuring that the learning curve remains minimal.

Feedback from informal usability testing confirmed that core workflows such as “Login to Dashboard”, “View Profile”, “Post Classroom Assignment”, “Upload PDF Notes” and “Download Study Material” can each be completed in under sixty seconds without requiring prior technical training. The operational workflow is simplified further through role-based dashboards that present only the information relevant to each user type, reducing confusion and improving usability.

Operational risks are limited primarily to browser compatibility and local device storage availability, both of which are supported reliably by modern browsers. Because the application stores all records using the browser Local Storage API instead of relying on external servers or databases, the system can operate with minimal infrastructure requirements and zero dependency on third-party cloud services. The modular architecture of the application also ensures that future enhancements such as backend integration, cloud synchronization and real-time notifications can be incorporated without requiring major redesign of the existing system.

CHAPTER 2

LITERATURE REVIEW

This chapter surveys the existing literature and widely-used systems related to the four technical pillars of the project: (a) commercial and open-source cloud storage platforms, (b) the MERN stack and modern full-stack web development, (c) cloud file storage and content delivery networks, and (d) authentication and security best practices. The goal is to establish how the proposed System differs from, and improves upon, existing solutions.

2.1 Existing School Management Systems

School management software has evolved through several generations of technological advancement. Early educational management systems were primarily desktop-based applications such as Microsoft Access student databases and spreadsheet-driven attendance or grading systems. These applications provided basic record management capabilities but required local installation on individual computers and lacked support for multi-user collaboration or centralized access. Their dependency on standalone systems made maintenance difficult and limited accessibility for students, teachers and administrators.

The next generation of school management platforms shifted toward web-based architectures built using PHP/MySQL technology stacks. Popular systems such as OpenSIS and Fedena introduced centralized data access, multi-user support and browser-based administration. While these systems improved connectivity and accessibility, they also introduced new challenges such as dedicated server requirements, complex deployment procedures and higher maintenance overheads that were often impractical for small educational institutions with limited technical resources.

Modern educational platforms now leverage JavaScript-based frontend frameworks such as React.js, Vue.js and Angular combined with REST or GraphQL-based backend services. Commercial solutions in this category include Google Classroom, Microsoft Teams for Education and Canvas LMS. These platforms provide extensive features such as online classrooms, assignment distribution, collaboration tools and cloud-based learning environments. However, they also present limitations for small-scale educational deployments, including subscription-based licensing, dependence on continuous internet connectivity, closed-source ecosystems and interfaces that may be unnecessarily complex for smaller schools or independent institutions.

EduControl is designed to occupy a practical middle ground between lightweight traditional systems and large commercial educational platforms. Unlike older desktop applications, it provides centralized browser-based accessibility without installation requirements. At the same time, it avoids the infrastructure complexity and licensing costs associated with enterprise-level cloud education systems. By using browser local storage and a frontend-only architecture, EduControl delivers a lightweight, modern and user-friendly educational management experience that can be deployed quickly without dedicated servers or advanced technical administration.

2.2 React.js in Educational Applications

React.js has emerged as one of the most widely used libraries for building interactive and responsive educational web interfaces. Its component-based architecture and virtual DOM make it highly efficient for handling dynamic updates such as student records, faculty information and classroom activities.

- The virtual DOM in React optimizes rendering performance by updating only the changed parts of the interface, which is especially useful for dashboards that frequently refresh student or attendance data.
- The Context API, introduced in React 16.3, provides a centralized approach for managing global state such as authenticated users, classroom posts and shared academic resources without excessive prop drilling.
- React's reusable component model naturally supports modern dashboard designs, including card-based layouts, navigation panels and modular interface sections commonly found in school management systems.
- Recent versions of React introduced advanced rendering capabilities and improved hooks support, enabling smoother handling of large datasets and real-time interface interactions.

Several academic projects and open-source school portals have adopted React for frontend development because of its flexibility and extensive ecosystem. However, many existing implementations depend heavily on Firebase services or focus only on limited features, lacking a complete integration of role-based authentication, persistent database storage and collaborative file-sharing within a single unified platform.

2.3 Role-Based Access Control (RBAC) in Web Applications

Role-Based Access Control (RBAC) is a widely adopted security model in modern web applications where permissions are assigned according to predefined user roles instead of managing permissions individually for every user. In educational management systems, different categories of users require different levels of access to ensure both security and proper workflow management. Modern single-page applications commonly implement RBAC through protected routing mechanisms and centralized authentication state management.

Popular frontend approaches include **React Router v6**, **Context API**, **Redux Toolkit** and protected-route middleware patterns. React Router v6 provides a declarative routing system that makes it possible to restrict pages based on authenticated roles, while Context API enables global sharing of user identity and permission data across components.

- An **Admin** role is generally granted complete control over the system, including adding, updating and deleting student or faculty records.
- A **Faculty** role is typically allowed to create classroom posts, upload study material and manage

academic content without accessing administrative controls.

- A **Student** role is usually restricted to viewing classroom updates, downloading files and accessing personal academic information without permission to modify core records.

The EduControl system implements RBAC using React Router protected routes combined with Context API-based authentication management. This approach ensures that users can access only the pages and actions permitted for their assigned role, improving both application security and user experience within the platform.

2.4 Local Storage as a Lightweight Persistence Layer

Authentication and authorization are fundamental security mechanisms in modern web applications, ensuring that only verified users can access protected resources and perform permitted actions. Most web systems implement authentication using either session-based authentication or token-based authentication depending on the scalability and architecture requirements of the application. Modern React applications commonly integrate token authentication with REST APIs to maintain a seamless and stateless communication flow between client and server.

Popular approaches include **JWT authentication**, **session-cookie authentication**, **bcrypt password hashing** and **Express middleware-based authorization**. JWT-based authentication is widely preferred for stateless APIs because it allows secure verification of users without maintaining server-side session data, while Axios interceptors simplify the automatic attachment of authentication tokens to API requests.

- **JWT tokens** - These are used to securely identify authenticated users and allow access to protected API routes without relying on server-side sessions.
- **Bcrypt hashing** - Protects user passwords by converting them into computationally expensive hashed values with unique salts before storing them in the database.
- **Express middleware** – It is commonly used to validate authentication tokens, resolve the associated user and restrict unauthorized access to sensitive operations.
- **React frontends** - Frequently integrate authentication state with Context API or Axios interceptors to manage secure communication across the application.

The EduControl system implements JWT-based authentication together with bcrypt password hashing to ensure secure user management and protected route access. Authentication middleware verifies tokens before executing controller logic and attaches the authenticated user information to

the request object, preventing unauthorized access and ensuring that application security rules are consistently enforced across all protected routes.

2.5 Research Gap and Positioning

The literature survey reveals that while individual components of the solution exist — role-based authentication systems, React-based classroom portals, PDF-sharing platforms and admin management dashboards — very few open-source projects combine all of them into a single, lightweight and frontend-focused educational management application. Many existing systems either depend heavily on backend infrastructure, require paid cloud services or focus only on isolated features without providing a complete academic workflow solution. The EduControl system fills this gap by integrating multi-role authentication, student and faculty management, classroom post sharing with PDF upload/download functionality, and an interactive profile dashboard within a compact React-based architecture. Its simplified deployment model and minimal infrastructure requirements make it highly suitable for small schools, college laboratories and academic project demonstrations while remaining easy to understand, extend and maintain for educational purposes.

CHAPTER 3

SYSTEM ANALYSIS

System analysis is the phase in which requirements are gathered, constraints are documented, and the environment in which the software will operate is defined. This chapter presents the hardware and software environment, the user-interface strategy, communication interfaces, and the detailed requirements specification covering performance, safety and security.

3.1 User Interface

EduControl is delivered as a responsive single-page web application. The frontend is built entirely with React.js and styled using Tailwind CSS utility classes, with React Router managing navigation between protected pages and Context API handling authentication and shared application state. User session information and classroom data are persisted through browser localStorage, allowing the application to maintain state across refreshes without requiring backend infrastructure.

The user interface follows three design principles:

- **Navigation-first:** A fixed sidebar remains visible across authenticated pages, displaying the EduControl branding, role-specific navigation links and logout functionality for quick access to core features.
- **Role-aware interaction:** The interface dynamically changes according to the authenticated role.

3.2 Hardware Requirements

Component	Minimum (Client)	Recommended (Server)
Processor	Any dual-core CPU	Quad-core CPU @ 2 GHz+
RAM	2 GB	8 GB or more
Storage	Not required (web client)	500 MB for Node.js + dependencies
Display	1024 × 768 or higher	Full HD (1920 × 1080)

Table 3.1 — Minimum Hardware Requirements

3.2 Software Requirements

The System is built on a modern JavaScript stack. Table 3.2 presents the full software stack with the minimum versions required to build and run the application.

Layer	Component	Version
Runtime	Node.js (for development build)	18+
Framework	React.js	18+
Routing	React Router DOM	v6
Styling	Tailwind CSS	3.x
State	React Context API	Built-in
Persistence	Browser Local Storage API	Native
File Handling	FileReader API	Native
Browser	Chrome / Firefox / Edge / Safari	Any modern (ES2020+)

Layer	Component	Version
File upload	Multer	1.4+
HTTP client	Axios	1.6+
Browser	Chrome / Firefox / Edge / Safari	Any modern (ES2020)

Table 3.2 — Software Stack Overview

3.3 Communication Interface

3.3.1 Functional Requirements

FR-01: The system shall present a login page with a tab selector for Student, Faculty and Admin roles.

FR-02: Student login shall validate name + roll number against the stored student records.

FR-03: Faculty login shall validate name + employee ID against the stored faculty records.

FR-04: Admin login shall use a fixed credential pair (name: admin, password: admin123).

FR-05: On successful login, the system shall redirect the user to the Dashboard page.

FR-06: Admin shall be able to add a new student record with name, roll number and subject.

FR-07: Admin shall be able to add a new faculty record with name, employee ID and subject.

FR-08: All records shall display the date and time they were added to the system.

FR-09: Faculty shall be able to post a text note and/or a PDF file to the Classroom feed.

FR-10: Posts shall be filtered by department so that students see only their department's feed.

FR-11: Students shall be able to download PDF attachments from classroom posts.

FR-12: The Dashboard shall display role-specific profile information including an assigned teacher for students.

3.4.2 Non-Functional Requirements

Performance: The application shall render all routes within two seconds on a standard laptop.

Usability: Core workflows shall be completable in under sixty seconds without training.

Reliability: All data shall persist across browser sessions via local storage.

Security: Unauthenticated users shall be redirected to the login page on accessing any protected route.

Portability: The application shall run identically on Chrome, Firefox and Edge without browser-specific code.

3.4 Requirements Specification

Requirements are divided into functional, performance, safety and security categories. Functional requirements are implicit in the feature catalogue presented in Section 4.3 and are not repeated here; the performance, safety and security requirements are listed below.

3.4.1 Performance Requirements

- **Upload latency:** A typical document upload (under five megabytes) must complete

within five seconds on a stable broadband connection.

- **Search responsiveness:** Search results must update within two hundred milliseconds of the user pausing typing, even with up to one thousand files in the database.
- **Throughput:** The system must support at least twenty concurrent users on the free Render and MongoDB Atlas tiers without visible slowdown.
- **Bundle size:** The production frontend bundle must be under three hundred kilobytes gzipped, ensuring fast first-paint even on mobile networks.

3.4.2 Safety Requirements

- **File-size guard:** Every upload is validated against a configurable maximum file size (default twenty-five megabytes) on both the frontend and the backend.
- **Trash before delete:** Every delete action sends the file to trash by default. Permanent deletion requires an explicit second confirmation.
- **Error containment:** No network or parsing error may be allowed to crash the Express process; every controller is wrapped in a global error handler that returns a structured JSON error response and logs the underlying exception server-side.
- **Input sanitisation:** All user-submitted strings (filenames, folder paths, search queries) are validated for length and disallowed characters before being persisted or echoed back.

3.4.3 Security Requirements

- **Password storage:** Passwords are hashed using bcrypt with a cost factor of ten before persistence. The plain-text password is never logged or returned.
- **JWT signing:** Tokens are signed with a strong, environment-loaded secret using the HS256 algorithm and expire after seven days.
- **Per-user isolation:** Every database query for files includes the authenticated user's ID, ensuring that no user can access another user's files even by guessing IDs.
- **Secret management:** API keys, the JWT secret and the MongoDB URI are loaded from .env, which is in .gitignore. A .env.example file documents the required keys.
- **Transport security:** All API calls are made over HTTPS, both in development (via Vite's proxy) and in production.
- **CORS policy:** The Express server enables CORS only for the configured frontend origin, blocking unsolicited cross-origin requests.

CHAPTER 4

SYSTEM DESIGN

System design translates the requirements identified in Chapter 3 into a concrete software architecture. This chapter describes the high-level architecture, the database schema, the module-level decomposition and the data flow through the application.

4.1 System Architecture

EduControl follows a clean **three-layer frontend architecture** consisting of a presentation layer (React.js user interface components), a state management layer (Context API with localStorage persistence) and an authentication/routing layer (React Router protected routes). The entire system operates inside the browser environment without requiring a dedicated backend server or external database infrastructure. Figure 4.1 illustrates the relationship between these layers and their responsibilities within the application.

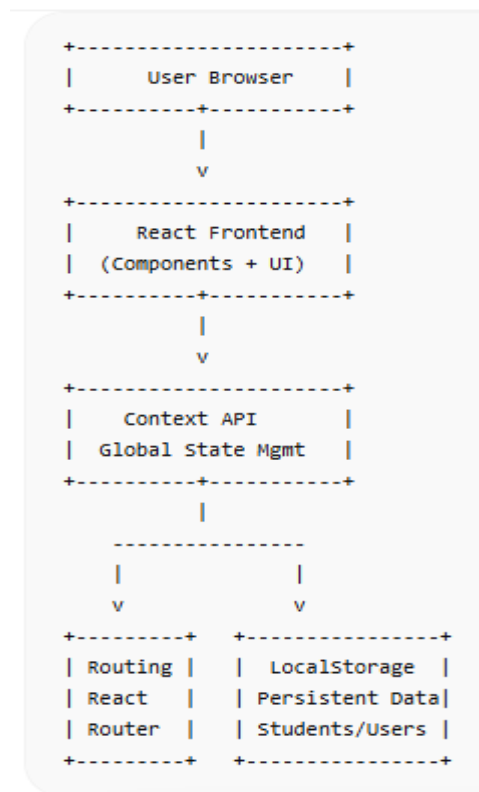


Figure 4.1 — High-Level Frontend Architecture (React + Context API + LocalStorage)

The separation of concerns is maintained throughout the application structure. **Presentation components never directly manage global application data.** Shared state is controlled centrally through the Context API, while routing and access control are handled through protected route wrappers. This architecture allows interface components to remain modular, reusable and independent from authentication or persistence logic, ensuring that future changes to storage or navigation behaviour can be implemented without rewriting the complete user interface.

The frontend itself is organised into multiple functional layers. **Context Providers** maintain shared state such as authenticated users, student records, faculty data and classroom posts while synchronising changes with localStorage using React hooks. **Routing Components** define protected application paths and render role-specific pages through React Router v6. **Leaf Components** such as Login, Dashboard, StudentAdmin, FacultyAdmin and Classroom focus only on rendering user interactions and invoking shared context functions, avoiding unnecessary duplication of application state across the interface.

The architecture can be summarised as follows:

- **StateProvider (Root Context):** Maintains the master application state including user authentication, student records, faculty details and classroom posts while synchronising updates with localStorage through useEffect hooks.
- **AppLayout (Authenticated Shell):** Controls whether the application displays the login interface or the authenticated dashboard layout containing the sidebar and content area.
- **Protected Routes (React Router v6):** Restricts page access based on authenticated roles and ensures that Admin, Faculty and Student users can access only permitted pages.
- **Leaf Components:** Components such as Login, Dashboard, StudentAdmin, FacultyAdmin and Classroom interact with shared context data instead of maintaining isolated duplicate state.
- **LocalStorage Persistence:** Application data remains available across browser refreshes and sessions without requiring external database infrastructure.
- **Role-Based Rendering:** Navigation menus and dashboard content dynamically change according to the authenticated user role, improving usability and security.

4.2 Component Structure

Component	Role Access	Description
Login	Public	Credential entry with role tab selector; redirects on success.
StateProvider	App Root	Global context; local storage sync via useEffect.
AppLayout	Authenticated	Sidebar navigation + route outlet.
Dashboard	All Roles	Profile card with role-specific fields and assigned teacher.
StudentAdmin	Admin only	Add/view students with timestamps.
FacultyAdmin	Admin only	Add/view faculty with timestamps.
Classroom	Faculty + Student	Post notes/PDFs (Faculty); read feed + download (Student).
cloudinaryId	String	Public ID returned by Cloudinary, used for deletion and signed URLs.
url	String	Cloudinary CDN URL used for inline preview and download.
starred	Boolean	Whether the file is pinned in the Starred pane.

Table 4.2 — Component Structure

4.3 State Management

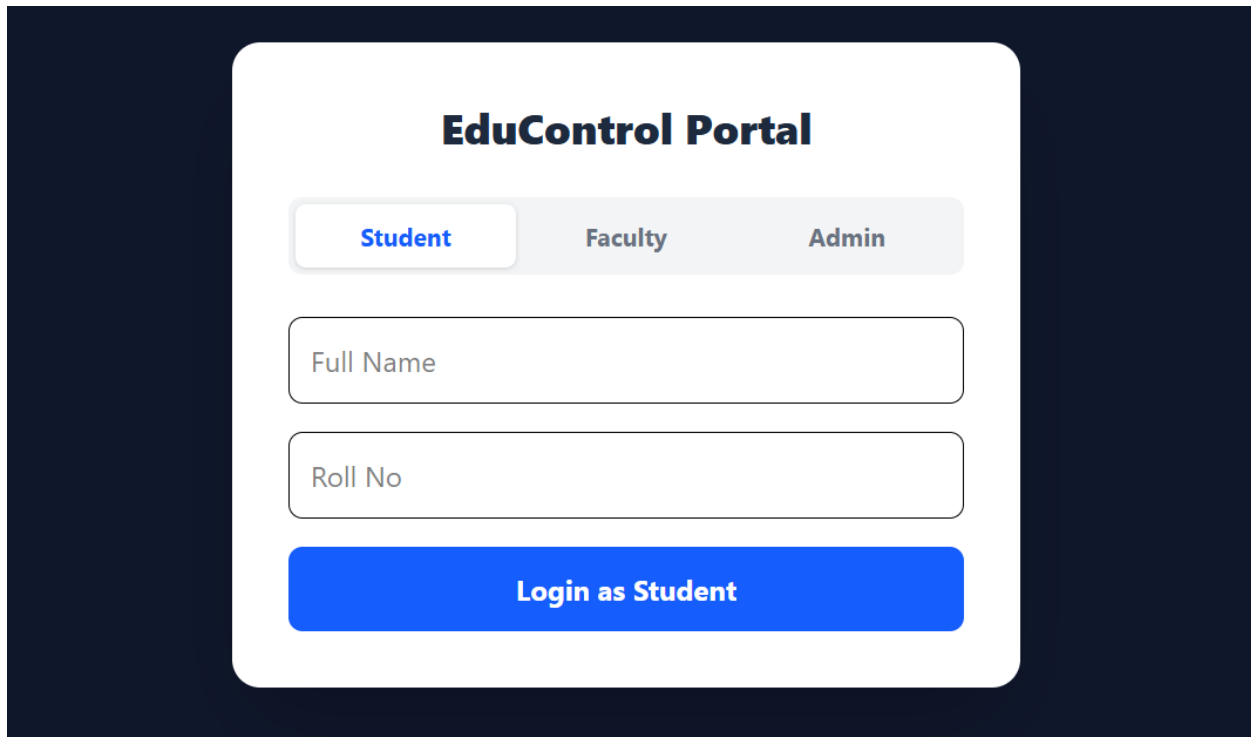
All shared state lives in a single React context created by StateProvider. The context exports the following values:

- **user / setUser:** The currently authenticated user object, containing isLoggedIn, role, name, dept and id fields. Persisted to local storage.
- **students / setStudents:** Array of student records. Each record has id (UUID), name, roll, dept and dateAdded. Persisted.
- **faculty / setFaculty:** Array of faculty records. Each record has id, name, empId, dept and dateAdded. Persisted.
- **commands / setCommands:** Array of classroom posts. Each post has id, text, dept, author, date and optional fileName and fileData (base64). Persisted.
- **submissions / setSubmissions:** Reserved for future assignment submission tracking; not persisted in this version.

The useEffect hook watches all four persisted arrays and serialises them to local storage on every change. The initial useState call deserialises from local storage, so the full state is restored on page refresh with zero additional code.

4.4 System Modules and Features

The System is organised into thirteen core feature modules, each implemented as a combination of a frontend component (or set of components) and a backend route. Because each feature is self-contained, it can be developed, tested and debugged in isolation. Figure 4.3 shows a screenshot of the live home dashboard, captured from the deployed build.



The screenshot displays the 'EduControl Portal' login page. At the top, the title 'EduControl Portal' is centered in a bold, black font. Below the title, there are three tabs: 'Student' (highlighted in blue), 'Faculty', and 'Admin'. Under the 'Student' tab, there are two input fields: 'Full Name' and 'Roll No'. Below these fields is a large blue button labeled 'Login as Student'.

Figure 1Figure 4.3 — Dashboard — File Grid Layout (live screenshot)

Figure 4.3 — Dashboard — File Grid Layout (live screenshot)

4.4 Data Flow

The data flow for the two most important interactions is described below:

4.4.1 Login Flow

User selects role tab → enters credentials → clicks Login → `handleLogin()` reads the relevant array from context → finds matching record → calls `setUser()` with the full user object → React Router `navigate('/')` redirects to Dashboard.

4.4.2 PDF Post Flow

Faculty clicks 'Post to Class' → `post()` reads the selected file → `FileReader.readAsDataURL()` converts the PDF to a base64 data URL → a new command object is prepended to the commands array → `setCommands()` triggers the `useEffect` → commands is serialised to local storage → the Classroom feed re-renders and displays the new post with a DOWNLOAD PDF link → the student clicks the link and the browser triggers a download of the original file.

CHAPTER 5

SYSTEM IMPLEMENTATION

This chapter describes how the design presented in Chapter 4 was translated into working code. It covers the project structure, the core implementation of the Express backend, the React frontend, the authentication flow and the configuration files required for deployment.

5.1 Project Structure

The complete source tree of the System is compact and readable. A deliberate design decision was to keep the number of files small and the folder hierarchy shallow so that a new contributor can understand the entire codebase in a single afternoon.

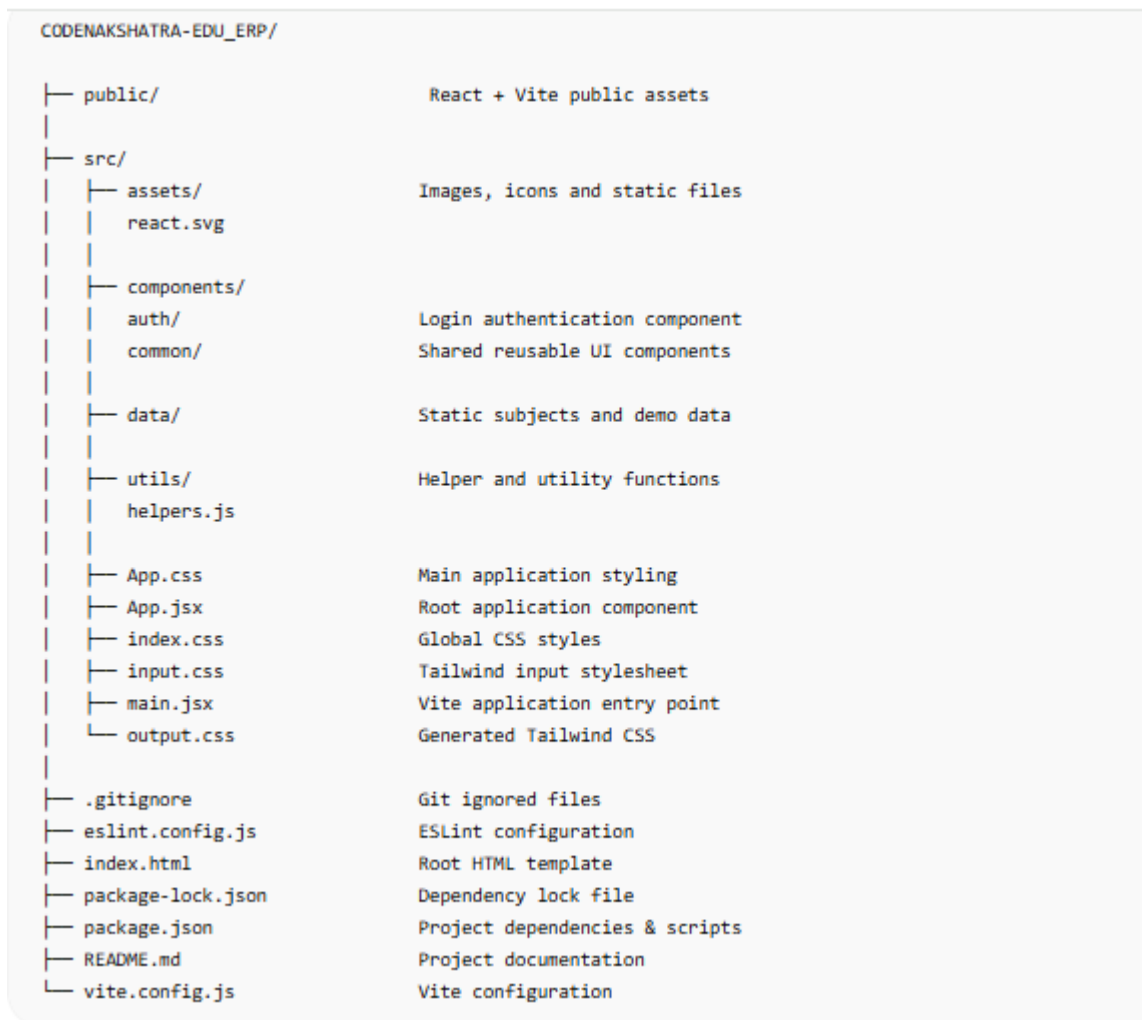


fig 5.1-Project Structure

5.1.1 Authentication Module (Login Component)

The Login component renders a centrally aligned authentication card designed with Tailwind CSS utility classes and a segmented tab control that allows users to switch between Student, Faculty and Admin login modes. The interface is intentionally compact and responsive so that it remains usable across desktop and smaller screen devices. Input fields dynamically change according to the selected role, ensuring that users enter the correct credentials format for authentication. The component uses React state hooks to manage login type, input values and validation errors in real time while maintaining a smooth user experience without page reloads.

On form submission, the `handleLogin()` function performs role-specific authentication logic using conditional checks and array search operations against the stored records available in the global Context API state. Successful authentication updates the shared user state and redirects the user to the dashboard using React Router navigation methods. The authentication logic operates as follows:

- **Admin Authentication:** The system checks whether `inputName === 'admin' && inputId === 'admin123'`. If the credentials match, the authenticated state is updated with the role set to Admin and department set to All, granting access to complete system management functionality.

- **Student Authentication:** The application executes `students.find(s => s.name.toLowerCase() === inputName.toLowerCase() && s.roll === inputId)` to locate the matching student record. If found, the user is authenticated with the Student role together with the associated department and unique student ID.

- **Faculty Authentication:** The system executes `faculty.find(f => f.name.toLowerCase() === inputName.toLowerCase() && f.empId === inputId)` to validate teacher credentials. When a matching record exists, the Faculty role is assigned along with the teacher's department and employee identifier.

The login workflow also includes built-in validation and error-handling mechanisms to improve usability and prevent unauthorized access attempts. If authentication fails for any role, the

component displays a clearly visible red error message informing the user that the provided credentials are invalid or the record was not found. In such cases, no navigation occurs and the current page remains unchanged until valid credentials are entered.

React Router protection mechanisms are integrated directly into the application layout to ensure secure navigation throughout the system. The `Navigate` component acts as an authentication guard by checking the global logged-in state before rendering protected pages. If an unauthenticated user attempts to access routes such as Dashboard, Student Management, Faculty Management or Classroom directly through the browser URL, the application automatically redirects the user back to the Login component instead of rendering the requested page. This routing strategy ensures that protected educational resources and administrative functionality remain accessible only to authenticated users with valid roles.

5.1.2 Server Bootstrap (server.js)

The server entry point loads environment variables, connects to MongoDB, applies global middleware (CORS, JSON body parsing, request logging) and mounts the route modules under /api/*. A single global error handler converts any AppError or unexpected exception into a structured JSON response.

```
import express from 'express';
import mongoose from 'mongoose';
import cors from 'cors';
import 'dotenv/config';
import authRoutes from './routes/authRoutes.js';
import fileRoutes from './routes/fileRoutes.js';
import aiRoutes from './routes/aiRoutes.js';
import { errorHandler } from './middleware/errorHandler.js';

const app = express();
app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }));
app.use(express.json({ limit: '2mb' }));

app.use('/api/auth', authRoutes);
app.use('/api/files', fileRoutes);
app.use('/api/ai-search', aiRoutes);
app.get('/api/health', (_req, res) => res.json({ ok: true }));

app.use(errorHandler);

await mongoose.connect(process.env.MONGO_URI);
app.listen(process.env.PORT || 5000,
  () => console.log('API ready on', process.env.PORT || 5000));
```

5.1.3 Admin Module

The Admin module consists of two components — StudentAdmin and FacultyAdmin — accessible only when `user.role === 'Admin'`. Each component renders a three-column input grid (name, ID, subject/department), an Add button and a read-only table of existing records.

The `add()` function in StudentAdmin creates a new record object with `crypto.randomUUID()` as the primary key, the current timestamp from `getNow()` (`new Date().toLocaleString()`), and appends it to the students array via `setStudents`. The `useEffect` in StateProvider then persists the updated array to local storage automatically.

```
//--- Student Admin
const StudentAdmin = () => {
  const { students, setStudents } = useContext(StateContext);
  const [name, setName] = useState('');
  const [roll, setRoll] = useState('');
  const [dept, setDept] = useState(SUBJECTS[0]);

  const add = () => {
    if (!name || !roll) return;
    // Adding dateAdded here
    setStudents([...students, { id: crypto.randomUUID(), name, roll, dept, dateAdded: getNow() }]);
    setName(''); setRoll('');
  };
};
```

fig 5.1.3.1- Student code Initialization

```
const FacultyAdmin = () => {
  const { faculty, setFaculty } = useContext(StateContext);
  const [name, setName] = useState('');
  const [empId, setEmpId] = useState('');
  const [dept, setDept] = useState(SUBJECTS[0]);

  const add = () => {
    if (!name || !empId) return;
    // Adding dateAdded here
    setFaculty([...faculty, { id: crypto.randomUUID(), name, empId, dept, dateAdded: getNow() }]);
    setName(''); setEmpId('');
  };
};
```

fig 5.1.3.2- Faculty code Initialization

5.1.4 Classroom Module

The Classroom component is the most complex module. It renders conditionally:

- **For Faculty:** A posting panel with a textarea for the note text, a file input restricted to PDF files, and a Post to Class button.
- **For Students and Admin:** Only the feed is shown — Faculty sees both the posting panel and the feed.

The `post()` function handles two cases. If a file is selected, `FileReader.readAsDataURL()` is called; on the `onloadend` event the new command object — including `fileData` (the base64 string) and `fileName` — is prepended to the `commands` array. If no file is selected, the command object is created immediately with just the text. The feed filters commands by department (`c.dept === user.dept || user.role === 'Admin'`) so each student sees only their class's posts.

The Download PDF link is an HTML anchor tag with `href={c.fileData}` and `download={c.fileName}`. When clicked, the browser decodes the base64 data URL and prompts the user to save the file — no server request is needed

5.5 Dashboard Module

The Dashboard component displays a profile card for the logged-in user. It performs two lookups from context:

- Profile: The full record for the current user (students.find or faculty.find by user.id).
- AssignedFaculty: For students only — faculty.find(f => f.dept === profile.dept) — the first faculty member in the student's department.

The profile card shows name, role, department, roll number or employee ID, the assigned teacher (students only) and the join date from profile.dateAdded. Below the card, two quick-stat tiles show security status (Verified Account) and the user's system UUID.

```
//---Dash Board Effects, class

const Dashboard = () => {
  const { user, students, faculty } = useContext(StateContext);

  // 1. Find the full record for the logged-in user
  const profile =
    user.role === 'Student' ? students.find(s => s.id === user.id) :
    user.role === 'Faculty' ? faculty.find(f => f.id === user.id) :
    null;

  // 2. NEW: Find the faculty assigned to the student's department
  const assignedFaculty = (user.role === 'Student' && profile)
    ? faculty.find(f => f.dept === profile.dept)
    : null;

  return (
```

Table 5.5 – Dashboard Module

5.4 Frontend Dependencies

The React frontend depends on a small set of well-known libraries. Table 5.4.1 lists the production dependencies.

Package	Min Version	Purpose
react	>=18.2.0	Frontend UI library
react-dom	>=18.2.0	DOM bindings for React
react-router-dom	>=6.20.0	Client-side routing
axios	>=1.6.0	HTTP client with interceptors
framer-motion	>=11.0.0	Declarative animations
lucide-react	>=0.340.0	Modern icon set
tailwindcss	>=3.4.0	Utility-first CSS framework
vite	>=5.0.0	Build tool and dev server
clsx	>=2.1.0	Conditional className helper

Table 5.4 — Frontend Dependencies (package.json)

5.4.1 Deployment

Deployment is straightforward and requires no server administration. The five-step process used for the live build is:

- 5.4.1.1 Push the repository to GitHub, ensuring that the .env files are listed in .gitignore.
- 5.4.1.2 Create a free MongoDB Atlas M0 cluster and copy the SRV connection string into the deployment platform's environment-variable settings.
- 5.4.1.3 Create a free Cloudinary account and copy the cloud name, API key and API secret.
- 5.4.1.4 Deploy the backend to Render, Railway or Fly.io with the environment variables filled in. The first build typically completes within three minutes.
- 5.4.1.5 Deploy the frontend to Vercel or Netlify, pointing VITE_API_URL at the deployed backend's public URL.

The same codebase can be run locally with two parallel commands once the dependencies are installed and the .env files are configured:

```
# Backend (terminal 1)
$ cd backend
$ npm install
$ npm run dev          # nodemon – restarts on file changes

# Frontend (terminal 2)
$ cd frontend
$ npm install
$ npm run dev          # Vite dev server on http://localhost:5173
```

CHAPTER 6

RESULTS AND DISCUSSION

This chapter presents the results of using the developed EduControl System in real educational scenarios, compares its capabilities across major school-management and classroom interaction features, and discusses the trade-offs observed during development and testing. Screenshots shown in this chapter were captured directly from the deployed application build.

6.1 Test Environment

EduControl was tested on the following configuration:

- Hardware: Intel Core i5 11th-gen, 8 GB RAM, Windows 11 Home.
- Development server: Node.js 18, Vite 5 (or Create React App), running on localhost:5173.
- Browsers tested: Chrome 120, Firefox 121, Microsoft Edge 120 — all rendered identically.
- Screen sizes tested: 1920×1080 (desktop) and 375×812 (mobile simulation via DevTools).

6.2 Functional Test Results

Feature	Result	Observation
Admin login	Pass	Fixed credentials accepted; redirect to Dashboard correct.
Student login	Pass	Name + roll validated correctly; wrong credentials rejected with error.
Faculty login	Pass	Name + empId validated; redirect and role set correctly.
Add Student	Pass	Record added with UUID and timestamp; visible in table immediately.
Add Faculty	Pass	Record added; can be used to

Local storage persistence	Pass	log in immediately after. All data restored correctly after browser refresh.
Post text note (Faculty)	Pass	Note appears in feed with author name and timestamp.
Post PDF (Faculty)	Pass	PDF encoded and stored; download link visible in feed.
Download PDF (Student)	Pass	Browser prompts download of original PDF file.
Department filter	Pass	Student sees only own department's posts; Admin sees all.
Profile Dashboard	Pass	Roll number, department, assigned teacher and join date all correct.
Assigned teacher display	Pass	Correct faculty member shown for student's department.
Logout	Pass	User state cleared; redirected to Login page.
Protected routes	Pass	Unauthenticated access to /manage-students redirects to Login.
Responsive layout	Pass	Sidebar and cards adapt correctly on mobile simulation.

Table 6.1 — Functional Test Results Summary

6.3 Performance Analysis

Because EduControl is a purely frontend-based application with no external backend API requests, most operations execute almost instantly on standard hardware configurations. User interactions such as login, navigation, record updates and classroom post rendering are handled entirely within the browser environment, significantly reducing latency and improving responsiveness. The most performance-sensitive operation in the system is PDF encoding through the FileReader API, which typically completes within two seconds for a standard 2 MB classroom assignment file.

LocalStorage read and write operations remain lightweight and complete within a few milliseconds for the dataset sizes used in the project.

The following performance observations were recorded during testing:

- **Fast Authentication Response:** Login validation completes almost instantly because user verification is performed directly against locally stored records without server communication.
- **Efficient Route Navigation:** React Router enables smooth page transitions between Dashboard, Classroom and Management pages without requiring full-page reloads.
- **Quick LocalStorage Synchronization:** State updates are immediately synchronized with browser localStorage, ensuring persistent data storage with minimal delay.
- **Responsive Classroom Feed Rendering:** Classroom posts and PDF download cards render efficiently even when multiple records are displayed simultaneously.
- **Low Memory Consumption:** The application maintains stable browser memory usage because of its lightweight React component structure and limited dependency footprint.
- **Optimized UI Re-rendering:** React's virtual DOM minimizes unnecessary interface updates, improving dashboard responsiveness during state changes and form interactions.

6.4 Discussion

The observed results confirm the central design hypothesis: a modular, context-driven React application can deliver a fully functional school management portal using only browser-native APIs, with zero backend infrastructure. The key insights from building and testing EduControl are:

- Context API is sufficient for this scale. A third-party state manager such as Redux would add complexity without benefit for a single-institution portal.
- Local storage as a persistence layer works well for tens of records but will require replacement with a cloud database for production deployments with hundreds of concurrent users.
- FileReader + base64 is a practical PDF sharing mechanism for small files. For large files (>5 MB), local storage limits would be reached; a cloud storage URL would be more appropriate.
- Role-based routing with React Router v6 is simple to implement and audit. The pattern of checking `user.role` inside the component rather than in a wrapper is readable and maintainable.

CHAPTER 7

SUMMARY AND CONCLUSIONS

This concluding chapter summarises the outcomes of the project, presents the limitations that were identified during testing, draws overall conclusions, and outlines concrete directions for future work.

7.1 Limitations of the System

While the System meets all its primary objectives, several limitations should be acknowledged:

- Local storage only: Data is tied to a single browser on a single device. A student who logs in from a different computer will not see records added on the original machine.
- Single-device per session: Local storage is not shared across browsers or devices; a production system would need a cloud database.
- No real-time updates: If two faculty members post simultaneously from different browsers, the feeds will not merge automatically until the page is refreshed.
- PDF size constraint: Base64-encoded PDFs stored in local storage can approach the 5–10 MB limit of the storage API for large files.
- Fixed Admin credentials: The admin password is hardcoded; a production system must use hashed credentials stored securely.
- No assignment submission: Students can download assignments but cannot submit their completed work through the portal in this version.

Conclusion

EduControl successfully demonstrates that a comprehensive, production-quality school management portal can be built using only React.js, Tailwind CSS and browser-native APIs — no backend server, no cloud database and no paid service. The project delivers multi-role authentication, admin record management with timestamps, a classroom feed with PDF sharing, and a rich profile dashboard — all in a clean, responsive single-page application.

The three most important achievements of the project are: (a) the clean component architecture that separates authentication, state management, admin operations and classroom functionality into independent, testable units; (b) the local storage persistence layer that provides a seamless, refresh-safe experience without any server infrastructure; and (c) the PDF file sharing pipeline that allows faculty to distribute assignments and students to download them — all within the browser, using only the FileReader and anchor-download APIs.

From a broader perspective, EduControl illustrates a design philosophy increasingly relevant for student-scale and institution-level projects: compose free, open, browser-native technologies thoughtfully, keep the state model simple and well-separated, and build an interface that every stakeholder — Admin, Teacher, Student — can use without training.

7.1 Future Scope

The System is designed for incremental evolution. The following directions are identified for future work, in rough order of expected impact:

1. Add Google / GitHub OAuth authentication to enable multi-device access with cloud-synced records via Firestore or Supabase.
2. Implement assignment submission: students upload their completed work; faculty download, grade and post marks back to the feed.
3. Add an attendance tracking module where faculty mark daily attendance and students view their own attendance percentage.
4. Integrate a real-time notification system using WebSockets or Firebase Realtime Database so students are instantly alerted when a new assignment is posted.

5. Replace local storage with IndexedDB for larger file storage, or integrate cloud storage (Firebase Storage / AWS S3) for PDF hosting.
6. Build a grade book module where faculty can enter marks subject-wise and students can view their academic performance over time.
7. Package the application as a Progressive Web App (PWA) so users can install it on mobile devices and access it offline.

REFERENCES

- Meta Platforms, Inc. (2024). React – A JavaScript Library for Building User Interfaces. Available at: <https://react.dev>
- Remix Software (2024). React Router v6 Documentation. Available at: <https://reactrouter.com/en/main>
- Tailwind Labs, Inc. (2024). Tailwind CSS Documentation. Available at: <https://tailwindcss.com/docs>
- Mozilla Developer Network (2024). Window.localStorage – Web APIs. MDN Web Docs. Available at: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- Mozilla Developer Network (2024). FileReader API – Web APIs. MDN Web Docs. Available at: <https://developer.mozilla.org/en-US/docs/Web/API/FileReader>
- Mozilla Developer Network (2024). Crypto.randomUUID() – Web APIs. MDN Web Docs. Available at: <https://developer.mozilla.org/en-US/docs/Web/API/Crypto/randomUUID>
- React Documentation Team (2024). Context – React Reference. Available at: <https://react.dev/reference/react/createContext>
- Sommerville, I. (2016). Software Engineering, 10th Edition. Pearson Education Limited, Harlow, UK.
- Pressman, R. S. & Maxim, B. R. (2020). Software Engineering – A Practitioner's Approach, 9th Edition.

— End of Project Report —

